

学习日志 · 云计算转行

这份文件怎么用

- 1. 我按《云计算6个月转行计划》逐课带你走，每课只推一小步，不提前灌后面内容。
- 2. 你在 Ubuntu 里跑我给的命令，把输出原样贴回来（贴错、报错都行，报错更有价值）。
- 3. 我把解读写进本文件的「我的解读」里，把卡点写进「卡点」，再决定下一课。
- 4. 你随时可以在本文件里补自己的话。

教材： 云计算6个月转行计划.md

进度总览

周	课程	状态
W1 D1	第 1 课 · 先认清你的机器	进行中（待贴输出）
W1 D1-D2	第 2 课 · 装工具 + Docker	进行中
W1 D2	第 3 课 · 让服务能被连上（SSH + 固定 IP）	待开始
W1 D2	第 4 课 · 建立"搞坏了能回来"的底气（备份）	待开始
W1 D3	第 5 课 · 文件系统：Linux 把东西放哪	待开始
W1 D4	第 6 课 · 纯动手：不碰鼠标完成 20 个文件操作	待开始
W1 D5	第 7 课 · 查找：find / grep	待开始

面试题库（每课一份）

用法：只看【问题清单】逐题口述并勾选 - []，再翻【参考答案】对照。答案只记结构：结论 → 为什么 → 命令/证据 → 加分点。

标记：★★★ 必背（面试高频 + 生产真遇到，有外部依据） / ★★ 理解 / 🛠 生产必会（企业天天用、面试少问） / ☆ 非面试题。

规则：每课上限 30 题；每组按星级降序（★★★ → ★★ → 🛠 → ☆，同级按题号升序）；题号永不重排。

规范与依据库：[面试题库/题库规范.md](#) —— 每写一课先看它。

课	面试题库	题量	★★★ 必背	🛠 生产必会	☆ 非面试题
第 1 课	先认清你的机器	24	10	1	1
第 2 课	装工具 + Docker	41	17	7	2

第 1 课 · 先认清你的机器

日期：2026-09-20 **对应计划：**W1 D1（含第 0 步的形态确认） **目标：**搞清楚"我现在手上是什么机器"，并把"先看输出、再下结论"这个反射种下去。

这一课在解决什么

你以后会反复听到一句话：**运维接手一台机器，第一件事永远不是装东西，是先搞清楚它是谁。**

因为所有后续判断都依赖这几个事实：内核版本、内存多大、磁盘还剩多少、是不是跑在虚拟化里。这些不确定，后面出的每个错你都只能猜。

所以第 1 课我们一条业务命令都不敲，只做"体检"。

第 1 课只对应计划 1.7「环境验收标准」的第 1、2 条（系统确认、资源确认）。

剩下 7 条会在第 2~4 课补齐，其中 **第 8、9 条（形态专项、备份可用）必须在任何「故意搞坏」的练习之前做完。**

本课任务

任务 A：判断你是什么形态（2 条命令）

```
systemd-detect-virt
lsblk -f
```

预期输出长这样：

- `systemd-detect-virt` 输出 `none` → **物理机**（如果还装了 Windows，就是双系统）
- 输出 `kvm` / `vmware` / `oracle` (VirtualBox) / `microsoft` (Hyper-V) → **虚拟机**，这个单词就是你的宿主虚拟化平台
- `lsblk -f` 会列出磁盘和分区，重点看：
- 除 Ubuntu 的分区外，有没有 `ntfs` 分区（有 = 双系统）
- Ubuntu 根分区挂载点是 `/`，容量多少
- `FSTYPE` 列是 `ext4`

补充讲解 · `lsblk -f` 到底在说什么

这是你对「硬盘 = 一块地」这个比喻的追问，结论记在这里，以后回看。

一、比喻的对应关系（逐项对上号）

比喻里的东西	实际是什么	在输出里的位置
一整块地	一块物理硬盘（或虚拟磁盘）	NAME 里 不带数字 的那行，如 nvme0n1、sda
地被切成几块	分区（partition）	NAME 里 带数字 的那行，如 nvme0n1p3
每块地上盖什么房	文件系统（filesystem）	FSTYPE 列：ext4 / ntfs / vfat / swap
房子挂哪个门牌	挂载点（mountpoint）	MOUNTPOINTS 列：/、/boot/efi
房子能住多少	剩余容量、使用率	FSAVAIL、FSUSE%
房子的身份证号	UUID	UUID 列（写 /etc/fstab 时用它，别用设备名）

二、一份真实输出的样子（示例，你的会不一样）

双系统机器的典型输出（新版 util-linux 列名是 MOUNTPOINTS，老教程写 MOUNTPOINT，是同一回事）：

NAME	FSTYPE	FSVER	LABEL	UUID	FSAVAIL	FSUSE%	MOUNTPOINTS
nvme0n1							
└─nvme0n1p1	vfat	FAT32		7A3B-1C4D	511.7M	1%	/boot/efi
└─nvme0n1p2	ntfs			01D8C2A4F5B6E7C8			
└─nvme0n1p3	ext4	1.0		5f2a9c31-8d4e-4b21-9a77-3c1e6f0b2d55	68.2G	28%	/
└─nvme0n1p4	swap	1		a1b2c3d4-e5f6-7890-abcd-ef1234567890			[SWAP]

树形缩进就是“分成了什么”的答案：└─┐ 缩进在 nvme0n1 下面的，都是这块盘切出来的分区。数一数 p1 p2 p3 p4 就知道它被分成了 4 块。

- 设备名不带数字（nvme0n1 / sda / vda）= 整块盘；带数字（nvme0n1p3 / sda3）= 分区
- NVMe 固态叫 nvme0n1p3，SATA 盘或虚拟磁盘叫 sda3，只是接口不同，含义一样
- MOUNTPOINTS 为空 = 这块分区**没挂载**，系统现在用不上它（Windows 分区在 Ubuntu 里就是这种状态，或只读挂载）
- 看到 loop0 loop1...**不用管**：那是 snap 把文件当磁盘用，不是你真实的硬盘

三、必须分清：分区 ≠ 文件系统

- **分区 = 划地**：在分区表里登记“从第几 G 到第几 G 归我”
- **文件系统 = 盖房**：mkfs.ext4 才是在这块地上盖房
- 地可以空着不盖房——这时 FSTYPE 就是空的，你看到的是“有地无房”
- **挂载 = 装门牌**：把房子接到 / 这棵目录树上你才走得进去。有房没门牌（没挂载），你在 ls 里是看不见它的

四、lsblk 的边界（它不告诉你的四件事）

你想知道	lsblk 能吗	该用什么
还有多少 未分配空间 （黑地）	不能	sudo parted -l
分区表是 MBR 还是 GPT	不能	sudo fdisk -l（看 Disklabel type）
是固态还是机械	能（要加参数）	lsblk -d -o NAME,ROTA,SIZE，ROTA=0 固态、1 机械
每个分区 谁吃掉了空间	不能	df -h（分区级）、du -sh ~/projects/*（目录级）

记住这个分工：**lsblk 看“盘的物理布局”，df 看“挂载后的用量”**。排磁盘故障时两个都要跑。

五、跑完 lsblk -f 你要抓的三行

1. 根分区那一行（MOUNTPOINTS 是 / 的那行）：**容量和剩余**，抄进笔记
2. 有没有 ntfs 行：**有 = 双系统**，W2 破坏性练习前必须先做 Timeshift/快照
3. 整块盘的 NAME 和分区数量：第 1.5④ 节扩容时，要先知道 100G 那块在磁盘的什么位置

任务 B：给机器做体检（6 条命令）

cat /etc/os-release
nproc
free -h
df -h /
ip -c addr show
docker ps

预期看到什么：

命令	看懂哪个数字	健康的样子
cat /etc/os-release	VERSION=	26.04
nproc	一个整数	≥ 4（虚拟机建议给 4~6 核）
free -h	Mem: 行的 available 列	≥ 12G（24G 里分给 Ubuntu 的部分）
df -h /	Avail 列	≥ 60G
ip -c addr show	非 lo 网卡的 inet 那行	形如 192.168.x.x 或 10.x.x.x
docker ps	报什么错	现在报 command not found 是 正常的 （Docker 还没装，第 2 课才装）；这条是第 2 课验收项 3、4 的伏笔

我的实测输出（贴这里）

（待你贴回）

我的解读

（待你贴输出后填写）

卡点

（待记录）

本课要记进笔记的三件事

- systemd-detect-virt 一条命令就能判断虚拟化平台，比翻设置快
- free -h 要看的是 available 而不是 free（为什么，下节课讲）
- df -h 看空间，df -i 看 inode —— 两个都会满，这是后面排障的伏笔

第 2 课 • 装工具 + Docker

对应计划：W1 D1（1.1 工具清单）+ W1 D2（1.2 Docker / 镜像加速 / 日志轮转） **节奏建议：**分两个晚上。今晚只做任务 A；任务 B~E 明晚做。 **目标：**工具装齐、Docker 能用，并亲手踩一次“装完 Docker 还要重新登录”的坑。

这一课在解决什么

排障靠的是工具，不是记忆。所以 1.1 的清单要一次装齐 —— 第 3 周排 CPU/内存/磁盘故障用 htop/iotop/sysstat，第 2 月抓包用 tcpdump，第 3 月写脚本要用 shellcheck 检查质量。

Docker 是第 4 月整套 K8s 实验的地基。这一课的重点不是“装上”，而是中间那个著名坑：**加进 docker 组之后，不重新登录，docker ps 照样报权限错误**。我们故意踩一次再修好 —— 自己踩过的坑才记得住。

任务 A：装工具（对应 1.1）

```
sudo apt update && sudo apt install -y \  
  curl wget git vim tmux tree jq unzip shellcheck \  
  net-tools dnsutils iputils-ping traceroute mtr-tiny \  
  tcpdump telnet netcat-openbsd \  
  htop iotop sysstat lsof strace \  
  build-essential python3 python3-pip python3-venv \  
  ca-certificates gnupg lsb-release
```

装完必须补两步（不做第一步，sysstat 等于白装）：

```
# ① sysstat 默认不采数据；不打开，以后 sar 查历史永远是空的，你会以为它坏了  
sudo sed -i 's/^ENABLED="false"/ENABLED="true"/' /etc/default/sysstat  
sudo systemctl enable --now sysstat  
  
# ② 验证三件事  
systemctl is-active sysstat      # 期望 active  
sar -u 1 3                       # 期望能打出 CPU 使用率  
which shellcheck tmux jq         # 期望三条路径都打印出来
```

iotop 必须 sudo iotop 才能看到全部进程的 IO —— 普通用户只能看到自己的。
nmap 不在必装清单：它是扫描工具，只能扫自己的资产；扫别人网段在国内属违法行为。
如果 apt update 慢到影响心情，可以换国内源（第 2 月有专门的换源课）。注意 Ubuntu 24.04 起默认源文件是 /etc/apt/sources.list.d/ubuntu.sources（deb822 格式），旧教程里的 /etc/apt/sources.list 可能已经不存在，别照着旧教程改。

任务 B：装 Docker（官方 apt 源）

```
# ① 加官方 GPG 公钥（apt 靠它验证“这个包真的来自 Docker”）  
sudo install -m 0755 -d /etc/apt/keyrings  
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \  
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg  
sudo chmod a+r /etc/apt/keyrings/docker.gpg  
  
# ② 加官方 apt 源（$(...) 自动取你的系统代号，不用手写）  
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \  
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" | \  
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null  
  
# ③ 安装  
sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin  
  
# ④ 验证“服务本身是好的”  
sudo systemctl is-active docker  
sudo docker run --rm hello-world
```

为什么不用 `curl -fsSL https://get.docker.com | sudo sh`? 那等于把网上一个随时会变的脚本直接喂给 root 执行。能用、快，但不可审计、不可复现。**面试问"你怎么装 Docker"**，答官方 apt 源是加分项。

任务 C（本课重点）：docker 组权限坑，故意踩一次

第 1 步：先确认"服务是好的，是我没权限"

```
sudo docker ps      # 能输出空表 → daemon 正常
docker ps           # 报 permission denied while trying to connect to the Docker daemon socket
```

两条命令的反差告诉你：**报错说的是"权限"，不是"服务没起来"**。排障第一步永远是分清这两类。

第 2 步：把当前用户加进 docker 组

```
sudo usermod -aG docker $USER
```

第 3 步：不重新登录，立刻试 ← 坑在这里

```
docker ps           # 期望：还是 permission denied（别慌，这是对的）
id -nG              # 期望：里面没有 docker
getent group docker # 期望：这一行末尾有你的用户名
```

为什么：usermod 改的是 /etc/group 这份"名单"；而一个进程属于哪些组，是**登录那一刻**就写进进程凭证的。你现在的 shell 是登录时创建的，名单改完它也不知道。

第 4 步：修好（二选一）

```
newgrp docker      # 临时：另开一个带 docker 组的 shell，退出即失效
# 或：完全退出登录（图形界面注销 / ssh 断开重连）—— 推荐，最干净

docker ps          # 期望：正常输出空表，不再报错
id -nG             # 期望：这次里面出现 docker
```

安全认知（面试会问）：docker 组等价于 root。能起特权容器的人，把宿主机 / 挂进容器就拿到了 root shell。所以生产上"给某人加 docker 组"是发一张 root 通行证，不是"方便一下"。

任务 D：镜像加速 + 日志轮转（/etc/docker/daemon.json）

```
sudo mkdir -p /etc/docker
sudo tee /etc/docker/daemon.json > /dev/null <<'EOF'
{
  "registry-mirrors": [
    "https://<你的专属ID>.mirror.aliyuncs.com"
  ],
  "log-driver": "json-file",
  "log-opts": { "max-size": "50m", "max-file": "3" }
}
EOF
sudo systemctl daemon-reload && sudo systemctl restart docker
docker info | grep -A3 "Registry Mirrors"    # 验证加速器生效
```

专属加速地址怎么拿（免费）：阿里云控制台 → 容器镜像服务 ACR → 创建**个人实例** → 左侧「镜像加速器」→ 复制你账号**专属的** `https://xxxx.mirror.aliyuncs.com`。这串 ID 绑定你的账号，**不能抄别人的**，也别用公共地址（dockerproxy.com 已失效；mirror.ccs.tencentyun.com **只在腾讯云内网稳定**）。备选：`docker.m.daocloud.io`、`docker.mirrors.ustc.edu.cn`。

三条规则：

1. **只配 1 主 1~2 备**。Docker 按顺序试，源不通要等超时才切下一个，堆多了只会更慢。
2. **registry-mirrors 只代理 pull，不代理 push。**
3. **log-opts 只对新建容器生效**，已存在的容器不会变 —— 以后查"磁盘被日志吃满"要记得这条。

故意搞坏一次（安全的小演练）：把 daemon.json 里随便删掉一个逗号或引号，然后

```
sudo systemctl restart docker      # 会失败
systemctl status docker            # 看到 failed
sudo journalctl -xeu docker | tail -20 # 期望能看到 JSON 解析相关的错误原文
# 改回去，再 restart，恢复
```

这就是"看报错原文 → 定位 → 验证"的最小闭环，第 1 月的排障反射从这一课开始练。

任务 E：本课验收（对应 1.7 的第 3、4、5 条）

```
docker ps          # 免 sudo 不报错 → 验收 3 通过
docker run --rm hello-world      # 打印 Hello from Docker! → 验收 4 通过
docker run -d -p 8080:80 nginx    # 起一个 nginx
curl -I localhost:8080 | head -1  # 期望 HTTP/1.1 200 OK → 验收 5 通过
docker rm -f $(docker ps -q --filter ancestor=nginx) # 收尾：删掉它
```

双系统用户注意：**验证服务永远在 Ubuntu 里用 curl，不要在 Windows 里开浏览器**。两个系统不能同时开；而且服务器上本来就没有浏览器，`curl` 才是生产里的方式。

我的实测输出（贴这里）

（待你贴回：每个任务的输出整段贴，报错更有价值）

我的解读

（待你贴输出后填写）

卡点

（待记录）

本课要记进笔记的三件事

- **组权限的生效时机**：`usermod -aG` 只改名单（`/etc/group`），已登录的 shell 不会立刻拿到新组 —— 要重新登录或 `newgrp`
- **分清两类报错**：`permission denied`（权限问题）和 `service not running`（服务问题），排查方向完全不同
- `daemon.json` 是纯 **JSON**，不能写注释；改完必须 `daemon-reload + restart`，且 `log-opts` 只对新建容器生效

待开始

第 3 课 • 让服务能被连上（SSH + 固定 IP）

预告：装 `openssh-server`、`systemctl enable --now ssh`、记下 IP，并决定 NAT / 桥接 / 静态绑定。这一课是第 3 月巡检脚本、第 5 月跨机验证、第 6 月 CI/CD 的地基。

第 4 课 • 建立"搞坏了能回来"的底气（备份与快照）

预告：`/etc` 配置备份 + 快照 / Timeshift。这一课必须在任何"故意搞坏"的练习之前上完，否则第 2 周改坏 `sudoers` 就是真事故。